Light

Algorithms — Searching & Sorting

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Searching algorithms
2. Sorting algorithms
3. Comparing algorithms
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Algorithms: Searching and Sorting (1CP2/01 Topic 1: Computational Thinking)**. This topic covers the four standard algorithms you must know: linear search, binary search, bubble sort, and merge sort. You need to understand how each algorithm works, trace through examples, and compare their efficiency.

Exam questions typically ask you to **trace** through algorithms showing variable values, **describe** how algorithms work step by step, **compare** algorithms in terms of efficiency, and **state** when each algorithm is appropriate to use. Expect questions worth 2—6 marks requiring you to show working through trace tables or annotated lists.

Why searching and sorting matter

Searching and sorting are fundamental operations in computing. Almost every program needs to find data or organise it in some way:

- **Search engines:** Finding relevant web pages from billions of indexed pages requires efficient search algorithms
- **Databases:** Looking up customer records, finding products, and retrieving information all use searching

- **Organisation:** Displaying items in order (by price, date, name) requires sorting algorithms
- **Efficiency:** Choosing the right algorithm can make the difference between a program that runs in milliseconds versus hours
- **Exam relevance:** This topic appears in Paper 1 Topic 1, typically worth 6—12 marks across several questions including trace tables and comparisons

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 1.3: Standard algorithms. You must be able to follow, trace, and compare these algorithms.

- **Describe** how linear search works and when it is used — checking each item in turn until the target is found or the end is reached
- **Describe** how binary search works — repeatedly dividing a sorted list in half to find the target
- **State** that binary search requires the data to be sorted, while linear search does not
- **Describe** how bubble sort works — repeatedly comparing adjacent items and swapping them if in the wrong order
- **Describe** how merge sort works — dividing the list into halves, sorting each half, then merging them back together
- **Compare** algorithms in terms of efficiency — number of comparisons, number of passes, suitability for different data sizes

> **Specification link:** This topic covers Edexcel 1CP2 Paper 1 Topic 1.3 — Standard algorithms including linear search, binary search, bubble sort, and merge sort. You must be able to understand, trace, and evaluate these algorithms.

Toolkit

These are the key patterns, definitions, and code templates you need for searching and sorting questions.

Algorithm overview

Algorithm	Purpose	Key requirement	How it works
Linear search	Find an item	None — works on any list	Check each item from start to end
Binary search	Find an item	List must be **sorted**	Repeatedly halve the search area
Bubble sort	Sort a list	None	Swap adjacent items that are in wrong order
Merge sort	Sort a list	None	Divide into halves, sort, merge back

Linear search — Python template

```
def linear_search(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i          # Return position if found
    return -1                 # Return -1 if not found
```

Binary search — Python template

```
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid] == target:
            return mid        # Found at mid position
        elif data[mid] < target:
            low = mid + 1     # Search right half
        else:
            high = mid - 1    # Search left half
    return -1                 # Not found
```

Bubble sort — Python template

```
def bubble_sort(data):
    n = len(data)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if data[j] > data[j + 1]:
                # Swap adjacent items
                data[j], data[j + 1] = data[j + 1], data[j]
    return data
```

Merge sort — conceptual pseudocode

> **Note:** You do not need to write merge sort code in the exam. You need to understand and describe how it works.

```
MERGE_SORT(list):
    IF length of list <= 1 THEN
        RETURN list
    END IF

    Split list into left_half and right_half

    sorted_left = MERGE_SORT(left_half)
    sorted_right = MERGE_SORT(right_half)

    RETURN MERGE(sorted_left, sorted_right)

MERGE(left, right):
    result = empty list
    WHILE left is not empty AND right is not empty:
        IF first item of left <= first item of right THEN
            Add first item of left to result
            Remove first item from left
```

```
        ELSE
            Add first item of right to result
            Remove first item from right
        END IF
    END WHILE
    Add any remaining items to result
    RETURN result
```

Key terminology

Term	Meaning
Comparison	Checking if one value is equal to, greater than, or less than another
Swap	Exchanging two values so they change positions
Pass	One complete iteration through the data (or part of it)
Sorted	Data arranged in order (ascending or descending)
Divide and conquer	Breaking a problem into smaller sub-problems, solving each, then combining results
Efficiency	How well an algorithm uses resources (time, comparisons, memory)

1. Searching algorithms

Searching algorithms find a specific item within a collection of data. The two standard searching algorithms you must know are linear search and binary search.

1.1 Linear search

Linear search (also called sequential search) checks each item in a list one by one, starting from the first item, until it finds the target or reaches the end.

How it works:

1. Start at the first item in the list
2. Compare the current item with the target
3. If they match, return the position — the item is found
4. If they don't match, move to the next item
5. Repeat until the item is found or the end of the list is reached
6. If the end is reached without finding the target, the item is not in the list

Key features:

- Works on **any list** — sorted or unsorted
- Simple to understand and implement
- In the worst case, checks every item (if target is last or not present)
- Best case: target is the first item (1 comparison)
- Worst case: target is last or not present (n comparisons for n items)

> **Exam tip:** Linear search does NOT require the data to be sorted. This is a key difference from binary search.

1.2 Linear search step-by-step

Example: Search for the value **7** in the list [3, 8, 2, 7, 1, 9]

```
List: [3, 8, 2, 7, 1, 9] Target: 7 Step 1: Check index 0 [3, 8, 2, 7, 1, 9]
3 ≠ 7, move to next Step 2: Check index 1 [3, 8, 2, 7, 1, 9] 8 ≠ 7, move to
next Step 3: Check index 2 [3, 8, 2, 7, 1, 9] 2 ≠ 7, move to next Step 4:
Check index 3 [3, 8, 2, 7, 1, 9] 7 = 7, FOUND at index 3 Result: Found at
position 3 Comparisons made: 4
```

1.3 Binary search

Binary search is a more efficient searching algorithm that works by repeatedly dividing the search area in half. It uses a "divide and conquer" approach.

Key requirement: The data **must be sorted** for binary search to work.

How it works:

1. Find the middle item of the list
2. Compare the middle item with the target
3. If they match, the item is found
4. If the target is less than the middle, search the left half
5. If the target is greater than the middle, search the right half
6. Repeat with the new half until found or the search area is empty

Key features:

- **Requires sorted data** — will not work correctly on unsorted lists
- Much faster than linear search for large datasets
- Each comparison eliminates half of the remaining items
- Maximum comparisons ≈ $\log_2(n)$ for n items

> **Exam tip:** If asked "why can't binary search be used?", the answer is usually "the data is not sorted".

1.4 Binary search step-by-step

Example: Search for **23** in sorted list [5, 12, 17, 23, 38, 44, 77, 84]

```
List: [5, 12, 17, 23, 38, 44, 77, 84] Target: 23 0 1 2 3 4 5 6 7 (indices)
Step 1: low=0, high=7, mid=(0+7)//2 = 3 [5, 12, 17, 23, 38, 44, 77, 84]
data[3] = 23 23 = 23, FOUND! Result: Found at position 3 Comparisons made: 1
```

Example: Search for **44** in the same list

```
List: [5, 12, 17, 23, 38, 44, 77, 84] Target: 44 Step 1: low=0, high=7,
mid=3 [5, 12, 17, 23, 38, 44, 77, 84] 23 < 44, search RIGHT half Set low =
4 Step 2: low=4, high=7, mid=(4+7)//2 = 5 [-, -, -, -, 38, 44, 77, 84]
data[5] = 44 44 = 44, FOUND! Result: Found at position 5 Comparisons made: 2
```

1.5 Calculating the middle position

In binary search, finding the middle position correctly is essential. The formula uses integer division:

Middle position = (low + high) // 2
Where `//` is integer division (rounds down to whole number)
Examples:

- low = 0, high = 7: mid = (0 + 7) // 2 = 7 // 2 = **3**
- low = 4, high = 7: mid = (4 + 7) // 2 = 11 // 2 = **5**
- low = 0, high = 4: mid = (0 + 4) // 2 = 4 // 2 = **2**

Worked Example 1 — Linear search trace

Question: The list below contains six values. Use linear search to find the value 5. Show each comparison made. [3 marks]

`[8, 3, 5, 1, 9, 2]`

Model answer:
Compare index 0: 8 ≠ 5, not found **(1)**
Compare index 1: 3 ≠ 5, not found **(1)**
Compare index 2: 5 = 5, found at position 2 **(1)**

Exam tip: Show each comparison clearly. State what is being compared and the result of each comparison.

Worked Example 1b — Tracing linear search Python code

Question: The following Python code performs a linear search. Trace through the code for the list [4, 7, 2, 9] searching for the value 2. Show the value of `i` and the output. [3 marks]

```
data = [4, 7, 2, 9]
target = 2
for i in range(len(data)):
    if data[i] == target:
        print("Found at", i)
        break
else:
    print("Not found")
```

Model answer:

i	data[i]	data[i] == target?	Action
0	4	4 ≠ 2, False	Continue
1	7	7 ≠ 2, False	Continue
2	2	2 = 2, True	Print and break

Output: Found at 2 **(3)**

Exam tip: When tracing code, show each iteration of the loop and the values of key variables at each step.

Worked Example 2 — Binary search trace

Question: Use binary search to find the value 31 in the sorted list below. Show the values of low, high, and mid at each step. [4 marks]

`[7, 15, 22, 31, 45, 56, 68, 79]`

Model answer:

Step	low	high	mid	data[mid]	Action
1	0	7	3	31	31 = 31, FOUND

The value 31 is found at index 3 in one comparison **(4)**

Exam tip: Binary search can find items very quickly if they happen to be near the middle. Show all variable values at each step.

Worked Example 3 — Binary search with multiple steps

Question: Use binary search to find the value 68 in the sorted list. Show the values of low, high, and mid at each step. [4 marks]

`[7, 15, 22, 31, 45, 56, 68, 79]`

Model answer:

Step	low	high	mid	data[mid]	Action
1	0	7	3	31	68 > 31, search right
2	4	7	5	56	68 > 56, search right
3	6	7	6	68	68 = 68, FOUND

The value 68 is found at index 6 after 3 comparisons **(4)**

Exam tip: When target > mid value, set low = mid + 1 (search right). When target < mid value, set high = mid - 1 (search left).

Worked Example 4 — Why binary search cannot be used

Question: A programmer wants to search for a value in the list [34, 12, 89, 23, 56, 7]. Explain why binary search cannot be used on this list. [2 marks]

Model answer:
Binary search requires the data to be sorted **(1)**.
This list is not in ascending or descending order, so binary search would not work correctly **(1)**.

Exam tip: Always state clearly that binary search requires sorted data. This is the most common reason it cannot be used.

2. Sorting algorithms

Sorting algorithms arrange data into a specific order, typically ascending (smallest to largest) or descending (largest to smallest). The two standard sorting algorithms you must know are bubble sort and merge sort.

2.1 Bubble sort

Bubble sort works by repeatedly stepping through the list, comparing adjacent items and swapping them if they are in the wrong order. The largest values "bubble up" to the end of the list.

How it works:

1. Start at the beginning of the list
2. Compare the first two adjacent items
3. If they are in the wrong order, swap them
4. Move to the next pair and repeat
5. After one complete pass, the largest item is at the end
6. Repeat the process, ignoring the last sorted item(s)
7. Continue until no swaps are needed in a pass

Key features:

- Simple to understand and implement
- Works on any list — does not require sorted data
- Inefficient for large lists — many comparisons and swaps
- After each pass, one more item is in its final position
- Can be optimised: stop early if no swaps occur in a pass

Exam tip: In bubble sort, after pass 1 the largest item is in place, after pass 2 the two largest are in place, and so on.

2.2 Bubble sort step-by-step

Example: Sort the list [5, 3, 8, 1, 2] into ascending order

```
Original: [5, 3, 8, 1, 2] Pass 1: [5, 3, 8, 1, 2] → 5>3, swap → [3, 5, 8,
1, 2] [3, 5, 8, 1, 2] → 5<8, no swap [3, 5, 8, 1, 2] → 8>1, swap → [3, 5,
```

```
1, 8, 2] [3, 5, 1, 8, 2] → 8>2, swap → [3, 5, 1, 2, 8] After pass 1: [3,
5, 1, 2, 8] — 8 is in place Pass 2: [3, 5, 1, 2, 8] → 3<5, no swap [3, 5,
1, 2, 8] → 5>1, swap → [3, 1, 5, 2, 8] [3, 1, 5, 2, 8] → 5>2, swap → [3,
1, 2, 5, 8] After pass 2: [3, 1, 2, 5, 8] — 5, 8 in place Pass 3: [3, 1,
2, 5, 8] → 3>1, swap → [1, 3, 2, 5, 8] [1, 3, 2, 5, 8] → 3>2, swap → [1, 2,
3, 5, 8] After pass 3: [1, 2, 3, 5, 8] — 3, 5, 8 in place Pass 4: [1, 2,
3, 5, 8] → 1<2, no swap After pass 4: [1, 2, 3, 5, 8] — SORTED
```

2.3 Merge sort

Merge sort uses a "divide and conquer" approach. It splits the list into smaller sublists, sorts them, then merges them back together in order.

How it works:

1. **Divide:** Split the list into two halves
2. **Conquer:** Keep splitting each half until you have single items (a single item is already sorted)
3. **Merge:** Combine the sorted parts back together in order

The merge process:

1. Compare the first item of each sorted sublist
2. Take the smaller item and add it to the result
3. Repeat until all items are merged

Key features:

- Efficient for large datasets — much faster than bubble sort
- Consistent performance — always takes similar time regardless of initial order
- Requires additional memory to store sublists during merging
- Uses a "divide and conquer" strategy (repeatedly breaking the problem down)

> **Exam tip:** Merge sort is "divide and conquer" — divide into halves, sort each half, merge back together.

2.4 Merge sort step-by-step

Example: Sort the list [6, 3, 8, 2, 7, 1] using merge sort

```
DIVIDE phase — split until single items: [6, 3, 8, 2, 7, 1] ↓ split [6, 3,
8] [2, 7, 1] ↓ split ↓ split [6, 3] [8] [2, 7] [1] ↓ split ↓ split [6] [3]
[8] [2] [7] [1] MERGE phase — combine in sorted order: [6] [3] → merge → [3,
6] [3, 6] [8] → merge → [3, 6, 8] [2] [7] → merge → [2, 7] [2, 7] [1] →
merge → [1, 2, 7] [3, 6, 8] and [1, 2, 7] → final merge: Compare 3 and 1 →
take 1 → [1] Compare 3 and 2 → take 2 → [1, 2] Compare 3 and 7 → take 3 →
[1, 2, 3] Compare 6 and 7 → take 6 → [1, 2, 3, 6] Compare 8 and 7 → take 7 →
[1, 2, 3, 6, 7] Take remaining 8 → [1, 2, 3, 6, 7, 8] Final result: [1, 2,
3, 6, 7, 8]
```

2.5 How merging works in detail

The merge operation is the key part of merge sort. It combines two sorted lists into one sorted list:

```
Merge [3, 6, 8] and [1, 2, 7]: Left: [3, 6, 8] Right: [1, 2, 7] Result: [] ↑
↑ 3 > 1, take 1 Left: [3, 6, 8] Right: [2, 7] Result: [1] ↑ ↑ 3 > 2, take 2
```

```
Left: [3, 6, 8] Right: [7] Result: [1, 2] ↑ ↑ 3 < 7, take 3 Left: [6, 8]
Right: [7] Result: [1, 2, 3] ↑ ↑ 6 < 7, take 6 Left: [8] Right: [7] Result:
[1, 2, 3, 6] ↑ ↑ 8 > 7, take 7 Left: [8] Right: [] Result: [1, 2, 3, 6, 7] ↑
Right empty, take remaining 8 Final Result: [1, 2, 3, 6, 7, 8]
```

Worked Example 5 — Bubble sort first pass

Question: Show the state of the list [7, 2, 5, 3] after the first pass of bubble sort. Show each comparison and swap. [4 marks]

Model answer:
Compare 7 and 2: 7 > 2, swap → [2, 7, 5, 3] **(1)**
Compare 7 and 5: 7 > 5, swap → [2, 5, 7, 3] **(1)**
Compare 7 and 3: 7 > 3, swap → [2, 5, 3, 7] **(1)**
After pass 1: [2, 5, 3, 7] — 7 is now in its final position **(1)**

Exam tip: Show each comparison clearly, stating whether a swap occurred and the resulting list.

Worked Example 5b — Tracing bubble sort Python code

Question: The following Python code performs one pass of bubble sort on the list [6, 2, 8, 1]. Trace through the code showing the values of `j`, the comparison made, and the state of `data` after each iteration. [4 marks]

```python
data = [6, 2, 8, 1]
n = len(data)
for j in range(n - 1):
    if data[j] > data[j + 1]:
        data[j], data[j + 1] = data[j + 1], data[j]
print(data)
```

Model answer:

j	data[j]	data[j+1]	Comparison	Swap?	data after
0	6	2	6 > 2	Yes	[2, 6, 8, 1]
1	6	8	6 > 8	No	[2, 6, 8, 1]
2	8	1	8 > 1	Yes	[2, 6, 1, 8]

Output: [2, 6, 1, 8] **(4)**

Exam tip: In Paper 2, you may need to trace through sorting code. Show the state of the list after each iteration, not just at the end.

Worked Example 6 — Bubble sort complete trace

Question: Use bubble sort to sort the list [4, 1, 3, 2] into ascending order. Show the list after each pass. [4 marks]

Model answer:
Original: [4, 1, 3, 2]
After pass 1: [1, 3, 2, 4] — 4 in place **(1)**
After pass 2: [1, 2, 3, 4] — 3, 4 in place **(1)**
After pass 3: [1, 2, 3, 4] — no swaps needed **(1)**
Sorted list: [1, 2, 3, 4] **(1)**

Exam tip: If the question asks for "after each pass", show the complete list state after each full pass through the data.

Worked Example 7 — Merge sort divide phase

Question: Show how the list [5, 2, 8, 1, 9, 4, 7, 3] would be divided during the first two stages of merge sort. [3 marks]

Model answer:
Original: [5, 2, 8, 1, 9, 4, 7, 3]
Stage 1 — Split into two halves: [5, 2, 8, 1] and [9, 4, 7, 3] **(1)**
Stage 2 — Split each half again:
[5, 2] and [8, 1] **(1)**
[9, 4] and [7, 3] **(1)**

Exam tip: In the divide phase, keep splitting in half until you have single items. Each split should produce roughly equal halves.

Worked Example 8 — Merging two sorted lists

Question: Show how the sorted lists [2, 5, 7] and [1, 4, 8] would be merged into a single sorted list. [4 marks]

Model answer:
Compare 2 and 1: 1 is smaller, take 1 → Result: [1] **(1)**

Compare 2 and 4: 2 is smaller, take 2 → Result: [1, 2]
Compare 5 and 4: 4 is smaller, take 4 → Result: [1, 2, 4] **(1)**
Compare 5 and 8: 5 is smaller, take 5 → Result: [1, 2, 4, 5]
Compare 7 and 8: 7 is smaller, take 7 → Result: [1, 2, 4, 5, 7] **(1)**
Only 8 remains, take 8 → Final result: [1, 2, 4, 5, 7, 8] **(1)**

Exam tip: When merging, always compare the first item of each list and take the smaller one. Continue until both lists are empty.

3. Comparing algorithms

Different algorithms have different strengths and weaknesses. Understanding when to use each algorithm is an important part of computational thinking.

3.1 Comparing searching algorithms

Feature	Linear search	Binary search
Requires sorted data?	No	Yes — must be sorted
Best case comparisons	1 (target is first)	1 (target is middle)
Worst case comparisons	n (check all items)	$\log_2(n)$
For 1000 items (worst)	1000 comparisons	≈10 comparisons
For 1,000,000 items (worst)	1,000,000 comparisons	≈20 comparisons
Simplicity	Very simple	More complex
Best for	Small or unsorted lists	Large sorted lists

Why binary search is faster

Binary search eliminates **half** of the remaining items with each comparison. For a list of 1000 items:

- After 1 comparison: 500 items remain
- After 2 comparisons: 250 items remain
- After 3 comparisons: 125 items remain
- After 10 comparisons: 1 item remains (1000 → 500 → 250 → 125 → 63 → 32 → 16 → 8 → 4 → 2 → 1)

Linear search might need all 1000 comparisons; binary search needs at most 10.

3.2 Comparing sorting algorithms

Feature	Bubble sort	Merge sort
Method	Compare and swap adjacent items	Divide, sort halves, merge
Comparisons (worst case)	Many — approximately n^2	Fewer — approximately $n \times \log_2(n)$
For 1000 items	≈1,000,000 comparisons	≈10,000 comparisons
Memory usage	Low — sorts "in place"	Higher — needs extra space for sublists
Simplicity	Very simple to understand	More complex (uses recursion)
Best for	Small lists, nearly sorted data	Large datasets

3.3 When to use each algorithm

Use linear search when:

- The data is **not sorted**
- The list is **small** (under ~20 items)
- You only need to search **once**
- Simplicity is more important than speed

Use binary search when:

- The data is **already sorted**
- The list is **large**
- You need to search **many times**
- Speed is important

Use bubble sort when:

- The list is **small**
- The list is **nearly sorted** already
- Memory is very limited
- Code simplicity is important

Use merge sort when:

- The list is **large**
- Consistent performance is needed
- Speed is more important than memory

- The data might be in any order

3.4 Efficiency terminology

When discussing algorithm efficiency, examiners expect specific terminology:

Term	Meaning	Example usage
Comparisons	Number of times items are compared	"Binary search makes fewer comparisons than linear search"
Passes	Complete iterations through the data	"Bubble sort may need n-1 passes"
Swaps	Number of times items change position	"Bubble sort makes many swaps"
Memory	Additional storage space required	"Merge sort requires extra memory for sublists"
Scalability	How performance changes with data size	"Binary search scales well to large datasets"

Worked Example 9 — Comparing search algorithms

Question: A company stores 10,000 customer records in alphabetical order by surname. Explain why binary search would be more suitable than linear search for finding a customer. [3 marks]

Model answer:
Binary search is more efficient for large datasets **(1)**.
The data is already sorted alphabetically, which is required for binary search **(1)**.
Binary search would need a maximum of approximately 14 comparisons ($\log_2 10000 \approx 14$), whereas linear search could need up to 10,000 comparisons **(1)**.

Exam tip: For comparison questions, address: (1) the requirement (sorted/unsorted), (2) efficiency for the given data size, (3) number of comparisons if possible.

Worked Example 10 — Comparing sorting algorithms

Question: A programmer needs to sort a list of 1,000,000 items. Compare bubble sort and merge sort for this task. [4 marks]

Model answer:

Merge sort would be more efficient for this large dataset **(1)**.
Bubble sort makes approximately n^2 comparisons, so would need around 1,000,000,000,000 (one trillion) comparisons **(1)**.
Merge sort makes approximately $n \times \log_2(n)$ comparisons, which is around 20,000,000 comparisons **(1)**.
However, merge sort requires additional memory to store sublists during the divide and merge phases, whereas bubble sort sorts in place **(1)**.

Exam tip: Always mention both advantages and disadvantages when comparing algorithms. Merge sort is faster but uses more memory.

Worked Example 11 — Algorithm selection

Question: A teacher has a list of 15 student names in random order and needs to find one specific student. Explain which search algorithm would be most appropriate. [3 marks]

Model answer:
Linear search would be most appropriate **(1)**.
The data is not sorted, so binary search cannot be used **(1)**.
For a small list of only 15 items, linear search is simple and efficient enough — sorting the data first to use binary search would take longer than just using linear search **(1)**.

Exam tip: Consider the overhead — for small unsorted lists, sorting first then using binary search is slower than just using linear search directly.

Worked Example 11b — Fixing a binary search error

Question: The following Python code is intended to perform binary search but contains an error. Identify the error and write the corrected line. [2 marks]

```
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    while low <= high:
        mid = (low + high) / 2      # Line with error
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Model answer:
Error: Line 5 uses `/` (float division) instead of `//` (integer division) **(1)**.
Corrected line: `mid = (low + high) // 2` **(1)**
Explanation: The index must be a whole number. Using `/` would give a decimal like 3.5, which cannot be used as a list index.

Exam tip: Integer division (`//`) is essential when calculating indices. This is a common error to look out for in Paper 2.

Worked Example 12 — Extended comparison

Question: A library system stores details of 50,000 books. The librarian frequently searches for books by ISBN (a unique number). The ISBNs are stored in ascending order. Discuss the most appropriate search algorithm. [6 marks]

Model answer:
Binary search is the most appropriate choice:
The data is already sorted by ISBN, which is required for binary search to work **(1)**.
With 50,000 books, binary search would need a maximum of approximately 16 comparisons ($\log_2 50000 \approx 16$) **(1)**.
Linear search could need up to 50,000 comparisons in the worst case, making it much slower **(1)**.
Since the librarian searches frequently, the time savings of binary search are significant over many searches **(1)**.
The data is already sorted, so there is no overhead of sorting before searching **(1)**.
Binary search is more complex to implement, but the performance benefit for this large, sorted dataset makes it worthwhile **(1)**.

Exam tip: For extended response questions, cover multiple factors: data requirements, efficiency, practical context (frequency of use), and any trade-offs.

Worked examples summary

This topic includes 15 worked examples covering the key skills you need for exam questions on searching and sorting algorithms, including Python code tracing and debugging.

Example	Skill tested	Algorithm
1	Tracing linear search	Linear search

Example	Skill tested	Algorithm
1b	Tracing linear search Python code	Linear search (Python)
2	Binary search trace table — found quickly	Binary search
3	Binary search trace table — multiple steps	Binary search
4	Explaining why binary search cannot be used	Binary search
5	Bubble sort — showing first pass	Bubble sort
5b	Tracing bubble sort Python code	Bubble sort (Python)
6	Bubble sort — complete trace	Bubble sort
7	Merge sort — divide phase	Merge sort
8	Merge sort — merging two lists	Merge sort
9	Comparing search algorithms	Comparison
10	Comparing sorting algorithms	Comparison
11	Algorithm selection — small unsorted data	Selection
11b	Fixing a binary search error (Python)	Binary search (Python)
12	Extended comparison — real-world context	Extended response

Quick reference

Key terms

Term	Definition
Linear search	A search algorithm that checks each item in sequence from start to end until the target is found or the end is reached
Binary search	A search algorithm that repeatedly divides a sorted list in half, comparing the middle item to the target to determine which half to search next
Bubble sort	A sorting algorithm that repeatedly compares adjacent items and swaps them if they are in the wrong order, causing larger values to "bubble" to the end
Merge sort	A sorting algorithm that divides the list into halves, sorts each half, then merges the sorted halves back together in order
Divide and conquer	A problem-solving approach that breaks a problem into smaller sub-problems, solves each, then combines the results

Term	Definition
Comparison	An operation that checks whether one value is equal to, greater than, or less than another value
Swap	An operation that exchanges two values so they change positions in a list
Pass	One complete iteration through the data (or part of it) in a sorting algorithm
Efficiency	A measure of how well an algorithm uses resources such as time and memory

Algorithm comparison summary

Algorithm	Type	Requires sorted data?	Best for	Worst case
Linear search	Search	No	Small/unsorted lists	n comparisons
Binary search	Search	Yes	Large sorted lists	$\log_2(n)$ comparisons
Bubble sort	Sort	N/A	Small/nearly sorted lists	$\approx n^2$ comparisons
Merge sort	Sort	N/A	Large datasets	$\approx n \times \log_2(n)$ comparisons

Common exam scenarios

Scenario	Best algorithm	Reason
Unsorted list, find one item	Linear search	Cannot use binary search on unsorted data
Sorted list, find one item	Binary search	Much faster for sorted data
Small list (under 20 items), sort	Bubble sort	Simple and efficient enough for small data
Large list, sort	Merge sort	Much faster for large datasets
Nearly sorted data	Bubble sort	Finishes quickly if few swaps needed
Frequent searches on same data	Sort first, then binary search	Sorting overhead paid once, searches are fast

Key numbers to remember

Binary search maximum comparisons:

- 8 items → 3 comparisons ($2^3 = 8$)
- 16 items → 4 comparisons ($2^4 = 16$)
- 1,000 items → 10 comparisons ($2^{10} = 1024$)
- 1,000,000 items → 20 comparisons ($2^{20} \approx 1{,}000{,}000$)

Bubble sort passes: Maximum of n-1 passes for n items

Typical pitfalls

These are common mistakes students make in searching and sorting questions. Avoiding these errors will help you gain marks in the exam.

Pitfall 1 — Forgetting binary search needs sorted data.
Wrong: Using binary search on any list.
Right: Binary search **only works on sorted data**. If data is unsorted, use linear search or sort first.

Pitfall 2 — Wrong middle calculation.
Wrong: mid = (low + high) / 2 giving decimal values.
Right: Use **integer division**: mid = (low + high) // 2. The middle index must be a whole number.

Pitfall 3 — Updating low/high incorrectly.
Wrong: Setting low = mid or high = mid.
Right: Set low = mid + 1 (search right) or high = mid - 1 (search left). Otherwise you may check the same item repeatedly.

Pitfall 4 — Not showing enough working in traces.
Wrong: Just writing the final answer.
Right: Show **each comparison**, what was compared, and the result. Marks are awarded for working, not just the answer.

Pitfall 5 — Confusing bubble sort passes and comparisons.
Wrong: "3 comparisons means 3 passes".
Right: Each **pass** involves multiple **comparisons**. A pass is a complete iteration; comparisons happen within each pass.

Pitfall 6 — Forgetting bubble sort reduces comparisons each pass.
Wrong: Comparing all items in every pass.
Right: After pass 1, the last item is sorted. After pass 2, the last two are sorted. **Reduce the comparison range** each pass.

Pitfall 7 — Describing merge sort without mentioning both phases.
Wrong: "It splits the list into smaller parts."
Right: Merge sort has two phases: **divide** (split into halves) and **merge** (combine sorted halves). Describe both.

Pitfall 8 — Comparing only speed, not other factors.
Wrong: "Merge sort is always better because it's faster."
Right: Consider **all factors**: speed, memory usage, data requirements (sorted/unsorted), simplicity, data size.

Pitfall 9 — Using $\log_2$ without understanding it.
Wrong: "Binary search takes log n comparisons" without explanation.
Right: Explain that binary search **halves the search area** each time, so $\log_2(n)$ tells us how many times we can halve n.

Pitfall 10 — Forgetting merge sort needs extra memory.
Wrong: "Merge sort has no disadvantages."
Right: Merge sort **requires additional memory** to store sublists during dividing and merging. Bubble sort sorts "in place".

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.